

LORA PRACTICE INTELLIGENCE

ML Underground — пошук рідкісних, але дієвих практик донавчання LoRA

Команда

Саваріна Дарина, Козін Андрій, Авдеєнко Дмитро, Мигаль Максим

ПРОБЛЕМА ТА ІДЕЯ

Проблема

- Практик донавчає LoRA за розрізненими туторіалами
- Немає способу дізнатися, що **реально працює**
- Популярне не дорівнює ефективно

Ідея

- Зібрати реальні конфіги з GitHub
- Звірити з успіхом моделей на HuggingFace
- Виокремити рідкісні практики, що корелюють з успіхом

ЧОТИРИ КВАДРАНТИ — ЯДРО ПРОДУКТУ

Rare + Works — прихований самоцвіт

Рідкісне і працює. Те, що шукаємо.

Common + Works — база

Усі так роблять, і це працює.

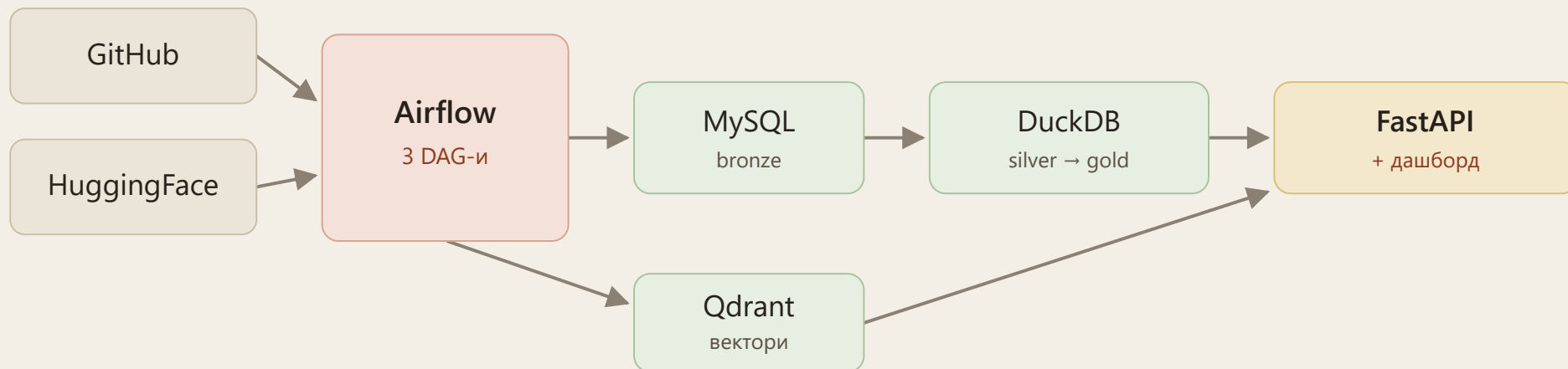
Common + Fails — cargo cult

Популярне, але неефективне.

Rare + Fails — глухий кут

Експериментальне і невдале.

ЗБІР І ОБРОБКА ДАНИХ



Прицільний збір з GitHub і HuggingFace, оркестрація Airflow, медальйон у DuckDB.

ВИТЯГ ПАРАМЕТРІВ

- Шари за пріоритетом: **AST** > **JSON** > **YAML** > **regex** > **LLM**; LLM-fallback (Claude Haiku) для нестандартних репо
- Variant D: параметри прямо з HF-карток — без зв'язування GitHub і HF

6 986	13 996	7 159	37%
репозиторіїв	HuggingFace-моделей	з параметрами	покриття витягу

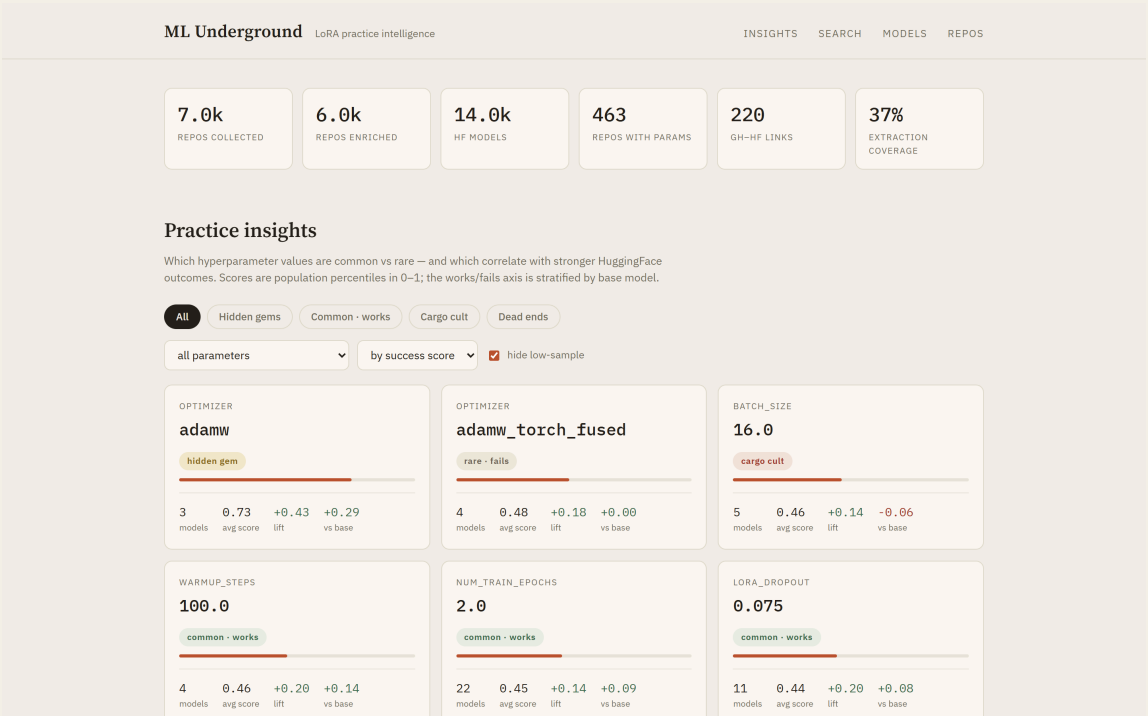
SUCCESS SCORE ТА КВАДРАНТИ

Композитний score (0–1)

- Перцентилі сигналів, зважена суміш:
- **0.55 завантаження + 0.35 лайки + 0.10 fan-out**

Надійність квадрантів

- Стратифікація по `base_model`
- `sample_ok` : щонайменше 3 моделі
- Кореляція, не каузація

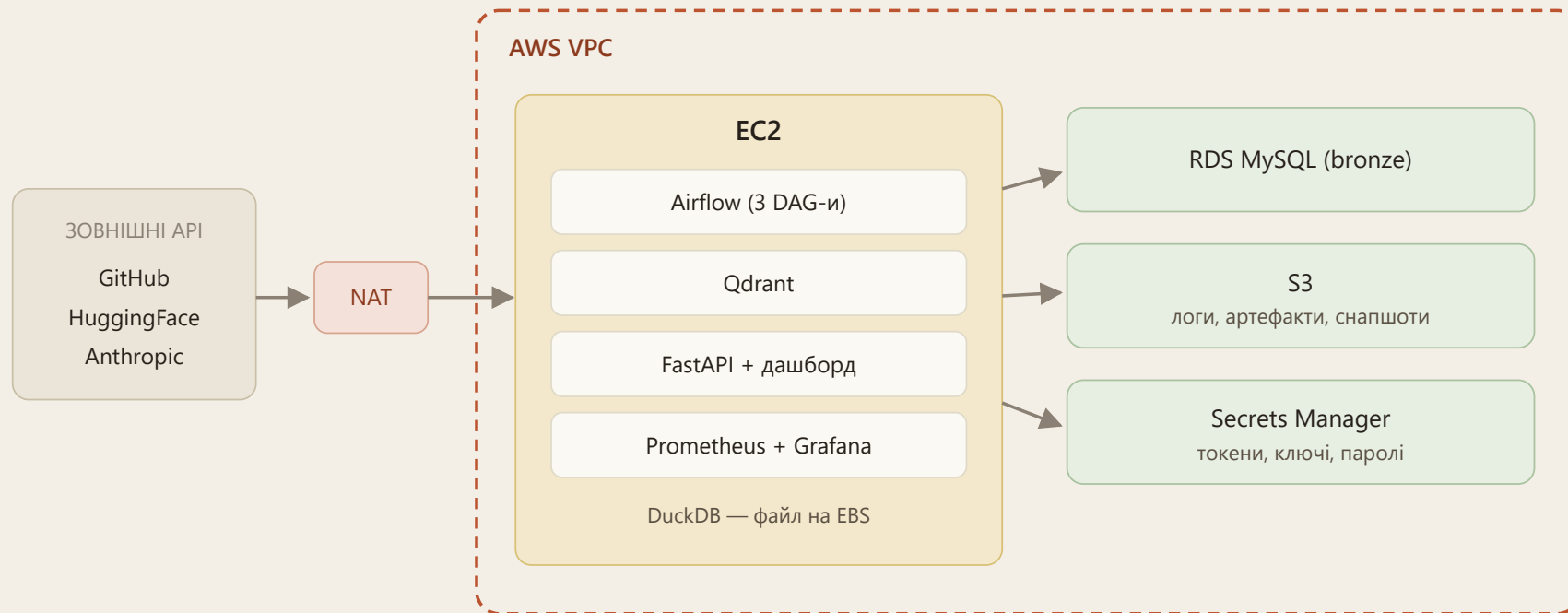


СЕМАНТИЧНИЙ ПОШУК І ASK



- Ембеддинги README і карток у Qdrant (all-MiniLM-L6-v2)
- Запит природною мовою → найближчі репозиторії та моделі
- Ask: відповідь Claude, заземлена в знайденому + факти warehouse
- Захист від prompt injection; чесне «недостатньо інформації»

РОЗГОРТАННЯ НА AWS



Уся інфраструктура — як код (Terraform); розкат через `terraform apply`.

ДЕПЛОЙ І ДАНІ

Міграція даних

- MySQL → RDS (`mysqldump`)
- Qdrant — снапшоти колекцій (14k+ векторів)
- DuckDB — похідний, регенерується
- Дані вже на S3

Образи і критерії

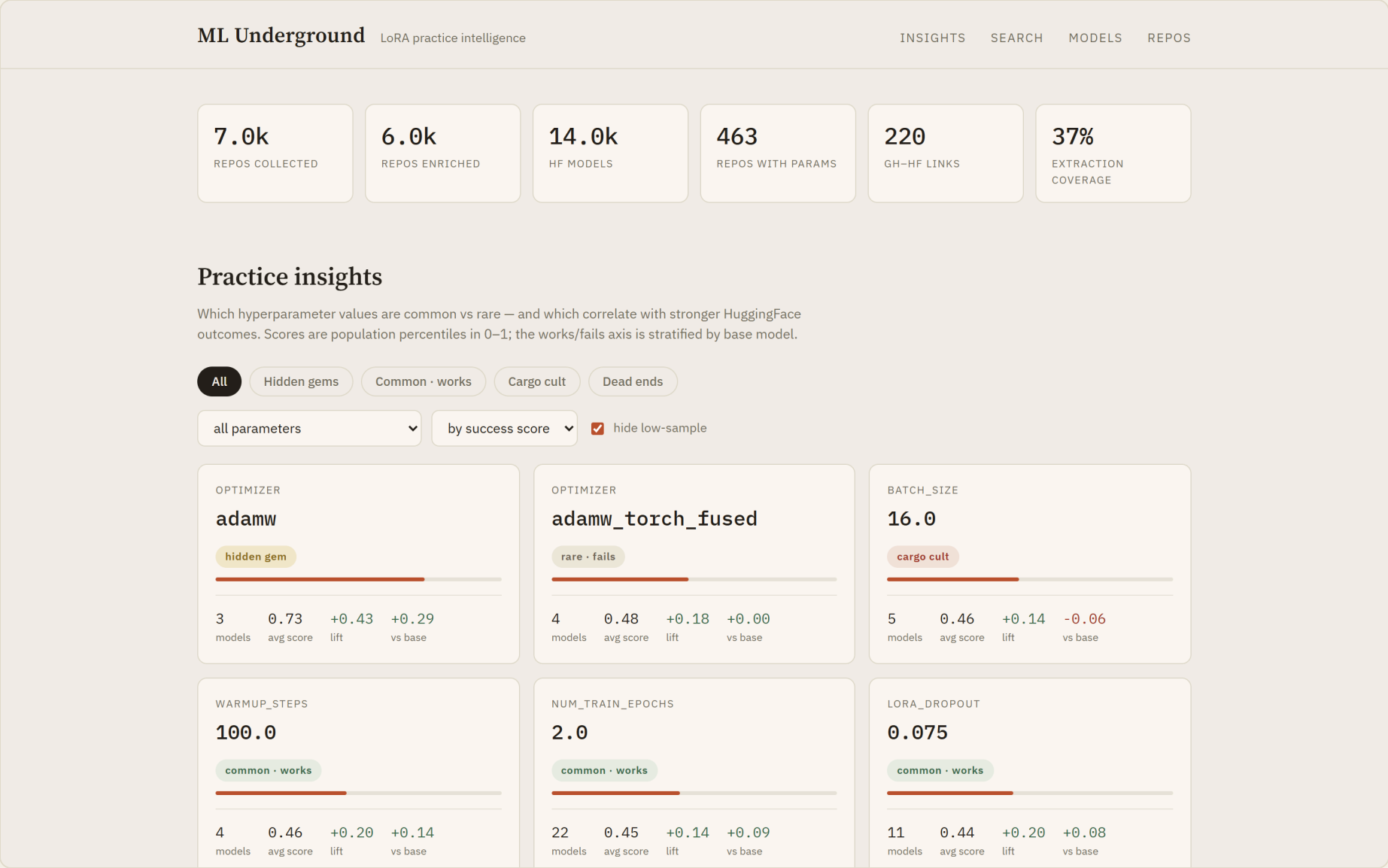
- ECR: Airflow, exporter, застосунок
- `terraform apply` без ручних кроків, усі три DAG-и зелені
- Повний деплой завершуємо окремим PR

ДЕМОНСТРАЦІЯ

`ml-underground.xyz`

Інсайти і квадранти, семантичний пошук, Ask із відповіддю на основі корпусу.

ДАШБОРД — ІНСАЙТИ



СЕМАНТИЧНИЙ ПОШУК

Semantic search over repo READMEs and model cards — or ask a question and get an answer grounded in the retrieved sources.

Search

Ask

REPO

bunmaskaaa/qlora-finetune

Python

sim 0.66

REPO

vishnudaspk/qwen7b-qlora-finetune-quantization-benchmarking

Python

sim 0.66

REPO

tzeivas/qlora-rs

Rust

sim 0.65

REPO

gautamvr/llms-1bit-quants-qlora

Python

sim 0.62

REPO

Nihalgk/DeepSeek-Coder-QLoRA-Fine-Tuning

Python

sim 0.62

REPO

zyx-stu/QLoRA_Finetuning_Cheaply

Jupyter Notebook

sim 0.62

REPO

EdwardNguyen2854/qlora-starter-kit

Python

sim 0.62

REPO

lweitkamp/qlora-dequantize_triton

Python

sim 0.62

REPO

leeroopedia/workflow-intel-ipex-llm-qlora-finetuning

Python

sim 0.62

ASK — ВІДПОВІДЬ З ДЖЕРЕЛАМИ

ML Underground

LoRA practice intelligence

INSIGHTS

SEARCH

MODELS

REPOS

Validated practice statistics (rare-but-works):

- `optimizer=adamw` — avg success **0.729** across 3 models
- `optimizer=paged_adamw_8bit` — avg success **0.375** across 11 models

So directly addressing your question about the 8-bit AdamW optimizer: `paged_adamw_8bit` does appear as a validated, working practice, but with a notably lower average success score (0.375) than plain `adamw` (0.729). However, note that `paged_adamw_8bit` was observed across a larger sample (11 models vs. 3), which is worth keeping in mind when weighing these scores — the higher-scoring `adamw` figure rests on a smaller sample.

Caveat on the rest of the corpus: None of the individual model cards or extracted recipes in the retrieved context (e.g., Megan1234/judgment-ai-adapter, devanshty/Code-Autopsy, laabamone/laabam-ai-3b-v1, Srushti-Thakur013/falcon-ecommerce-qa) specify their optimizer. They document rank, alpha, dropout, quantization, and learning-rate schedules, but not the optimizer. So I can't corroborate the practice statistics against any specific repo here.

Bottom line from the available data: Both `adamw` and `paged_adamw_8bit` are validated working choices. `paged_adamw_8bit` is the more commonly observed of the two (and is the standard pairing for QLoRA, since paging helps with the memory pressure of quantized training), while plain `adamw` carries the higher average success score on a smaller sample.

SOURCES

REPO

vishnudaspk/qwen7b-qlora-finetune-quantization-benchmarking

sim 0.60

Python

REPO

bunmaskaaa/qlora-finetune

sim 0.59

Python

REPO

EdwardNguyen2854/qlora-starter-kit

sim 0.58

Python

REPO

Exodussubhash/Synthetic-FineTuner

sim 0.57

Jupyter Notebook

REPO

HemGondaliya/llama2-qlora-qa

sim 0.57

Python

ДЯКУЄМО ЗА УВАГУ!

Саваріна Дарина, Козін Андрій, Авдеєнко Дмитро, Мигаль Максим